Project 4

EECE 5614: Reinforcement Learning and Decision Making Under Uncertainty

Shoukang Yu
Northeastern University

Spring 2026

1 Problem Setup and Parameters

We solve an 8×8 stochastic maze problem using three Deep Q-Network variants: **Standard DQN**, **Double DQN**, and **Dueling DQN**. The state is represented as a 2-dimensional vector $s = [x, y]^T$ where $x, y \in \{0, 1, \ldots, 7\}$, normalized to $[0, 1]$ as $\tilde{s} = [x/7,\ y/7]^T$ before being fed to the network. After each action, with probability $1 - P$ the agent moves to the anticipated state, and with probability $P/2$ each, it moves to one of the two perpendicular cells.

Table 1: Hyperparameters used in all experiments.

Parameter	**Value**
Replay memory size $\lvert\mathcal{D}\rvert$	10,000
Max episode length T_{epi}	50
Q-network update frequency N_{QU}	4
Discount factor γ	0.98
Minibatch size N_{batch}	64
Learning rate α	5×10^{-4}
Soft update rate η	1×10^{-3}
Number of episodes N_{epi}	3000
ϵ schedule	$\max(0.1,\ 0.999^k)$
Hidden layer width / # hidden layers	128 / 3
Transition noise P	0.025

State representation. The raw state is an integer coordinate pair $(x, y) \in \{0, 1, \ldots, 7\}^2$. Before being fed to the Q-network, each coordinate is normalized by dividing by GRID_SIZE $- 1 = 7$, yielding $\tilde{s} = [x/7,\ y/7]^T \in [0, 1]^2$. Normalization keeps the network input in a small, bounded range that is compatible with the default weight-initialization assumptions (Kaiming/Xavier), which leads to more stable gradients during training than feeding raw integer coordinates on the order of 0–7.

Epsilon schedule. We use an exponentially decaying ϵ with a floor of 0.1: $\epsilon_k = \max(0.1,\ 0.999^k)$, where k is the episode index (starting from $k = 1$). Early in training, $\epsilon \approx 1$ encourages broad exploration of the maze; as k grows, ϵ decays toward 0.1 so the agent increasingly exploits the learned greedy policy. The floor of 0.1 ensures a non-zero residual exploration rate, preventing the agent from getting permanently locked into a suboptimal action pattern.

Training process. The agent is trained for $N_{\text{epi}} = 3000$ episodes. Each episode starts from a uniformly *random non-goal state* — sampled from all walkable cells except the goal cell; the

blue start cell is *not* excluded and remains in the sampling pool. At each step, the agent selects an action using ϵ-greedy exploration, interacts with the environment, and stores the experience $(s, a, r, s', \text{done})$ in the replay buffer. Every $N_{QU} = 4$ steps, a minibatch of 64 experiences is sampled to update the Q-network by minimizing the MSE loss $L = \frac{1}{N_{\text{batch}}} \sum_i (z_i - Q^w(s_i, a_i))^2$, where $z_i = r_i + \gamma \max_{a'} Q^{w^-}(s'_i, a')(1 - \text{done}_i)$ for Standard/Dueling DQN, and uses the Double DQN target for the Double DQN variant. After each update, the target network weights are updated via soft update $w^- \leftarrow \eta w + (1 - \eta) w^-$.

2 Q2: Standard DQN — Avg Reward and Avg Loss

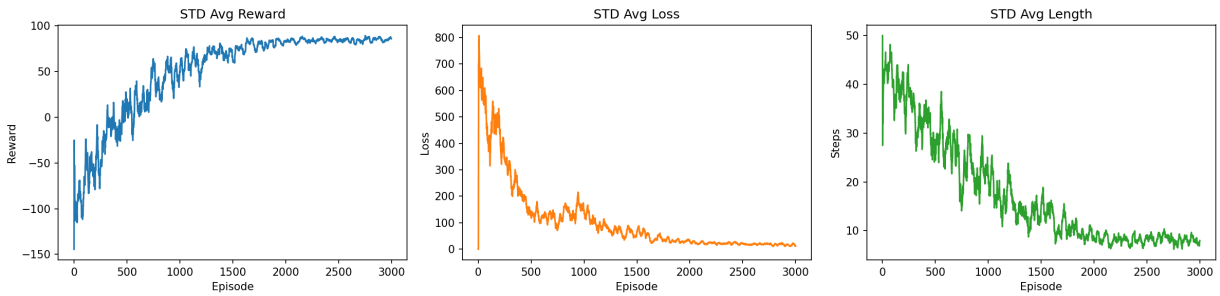


Figure 1: Standard DQN: average reward, average loss, and average episode length (rolling window = 25).

Key code — Standard DQN target computation: See Figure 2: the target is computed as $z_i = r_i + \gamma \max_{a'} Q^{w^-}(s'_i, a')(1 - \text{done}_i)$ via `target_net(s_next).max(dim=1)[0]`.

```
# Target z_i
with torch.no_grad():
    if self.mode == 'double':
        # Double DQN: Q-net selects action, target-net evaluates
        best_a = self.q_net(s_next).argmax(dim=1, keepdim=True)
        q_next = self.target_net(s_next).gather(1, best_a).squeeze(1)
    else:
        q_next = self.target_net(s_next).max(dim=1)[0]

    targets = r + self.gamma * q_next * (1.0 - done)
```

Figure 2: Standard DQN core update implementation.

Discussion: The Avg Reward (left panel) starts near -130 and rises monotonically, crossing zero around episode 400 and plateauing near 80–90 after episode 1500. This reflects the transition from random exploration ($\epsilon \approx 1$) to an increasingly greedy policy that reliably reaches the goal. The Avg Loss (center panel) peaks sharply near episode 50 at roughly 800, then decays to below 20 by episode 3000. The early spike occurs because target values vary widely when the network is untrained; as Q-values converge the TD error shrinks and loss falls accordingly.

3 Q3: Standard DQN — Obtained Policy

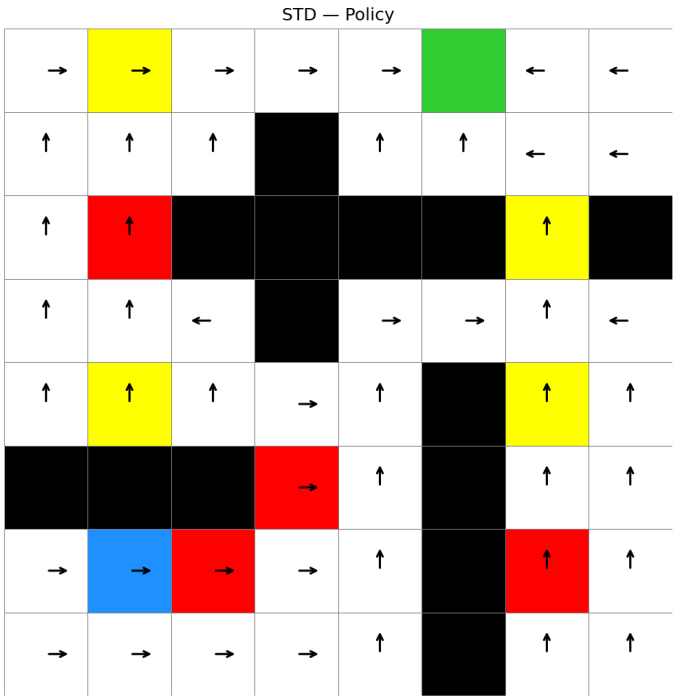


Figure 3: Standard DQN: learned greedy policy. Arrows indicate the best action at each state.

Discussion: The learned policy (Figure 3) is qualitatively optimal: arrows in the top rows point toward or directly into the goal at $(5, 0)$, while arrows in the lower rows consistently direct the agent upward and rightward, routing around the wall cluster at rows 2–3. Penalty cells are handled correctly — the agent at yellow cell $(6, 2)$ points upward and the agent at red cell $(1, 2)$ also points upward rather than remaining in the penalty region.

4 Q4: Standard DQN — State Values

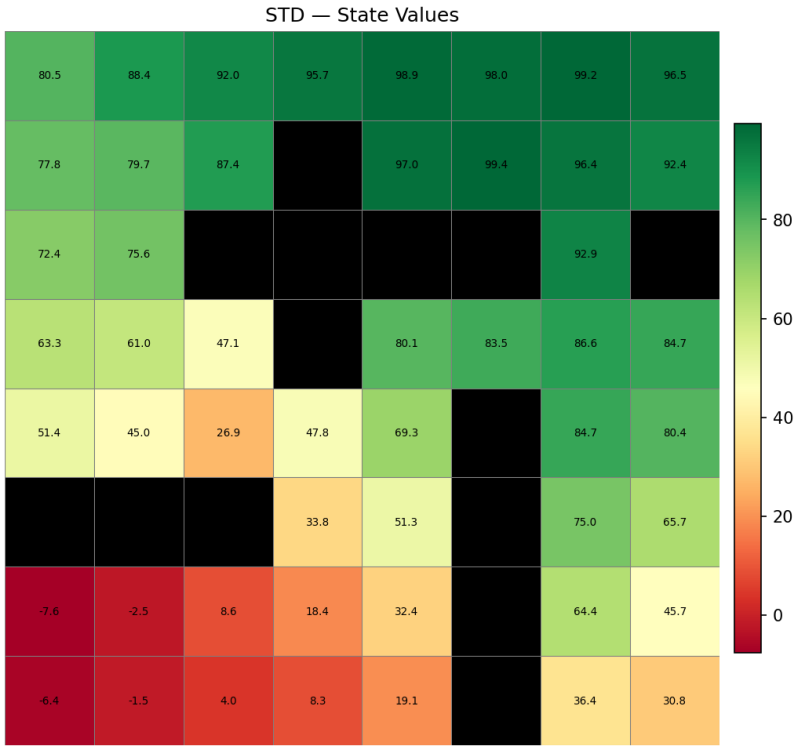


Figure 4: Standard DQN: state values $V(s) = \max_a Q(s, a)$. Green = high value, red = low value.

Discussion: The heatmap shows a clear spatial gradient consistent with maze geometry. States in row 0 near the goal have the highest values (95–99), while the bottom-left corner (rows 6–7, cols 0–1) reaches -7.6 and -6.4. These negative values arise because those states are far from the goal, adjacent to red penalty cells (reward -10), and require many steps to escape. The gradient decreases smoothly with distance from the goal, modulated by wall obstacles and penalty cells.

5 Q5: Standard DQN — Path from Start

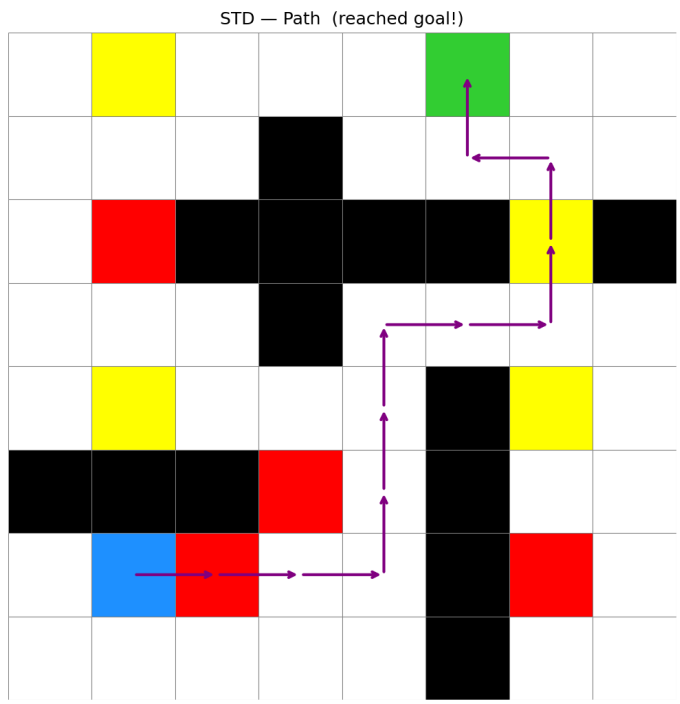


Figure 5: Standard DQN: path from start (blue) to goal (green) following the learned greedy policy.

Discussion: Starting from the blue cell $(1, 6)$, the greedy policy moves right along row 6 to column 4, then turns upward through column 4 to row 3, proceeds right to column 6, continues upward through the yellow cell $(6, 2)$ to row 1, steps left to $(5, 1)$, and finally moves up to reach the goal at $(5, 0)$. The agent successfully avoids all red penalty cells and the large wall cluster, reaching the goal in 12 steps.

6 Q6: Standard DQN — Avg Length

The Avg Length during training is shown in the rightmost panel of Figure 1 (Section 2).

Discussion: The Avg Length curve starts near the episode cap of 50 steps and decreases steadily, falling below 15 by episode 1500 and stabilizing around 5–10 by episode 3000. This mirrors the Avg Reward curve inversely: a better-trained agent reaches the goal faster, accumulating fewer -1 per-step penalties and thus earning higher total reward. Both curves converge at approximately the same episode, confirming that reward improvement is driven by path efficiency.

7 Q7: Effect of Learning Rate α

We compare three learning rates: $\alpha/10 = 5 \times 10^{-5}$, $\alpha = 5 \times 10^{-4}$ (baseline), and $\alpha \times 10 = 5 \times 10^{-3}$, with all other hyperparameters fixed.

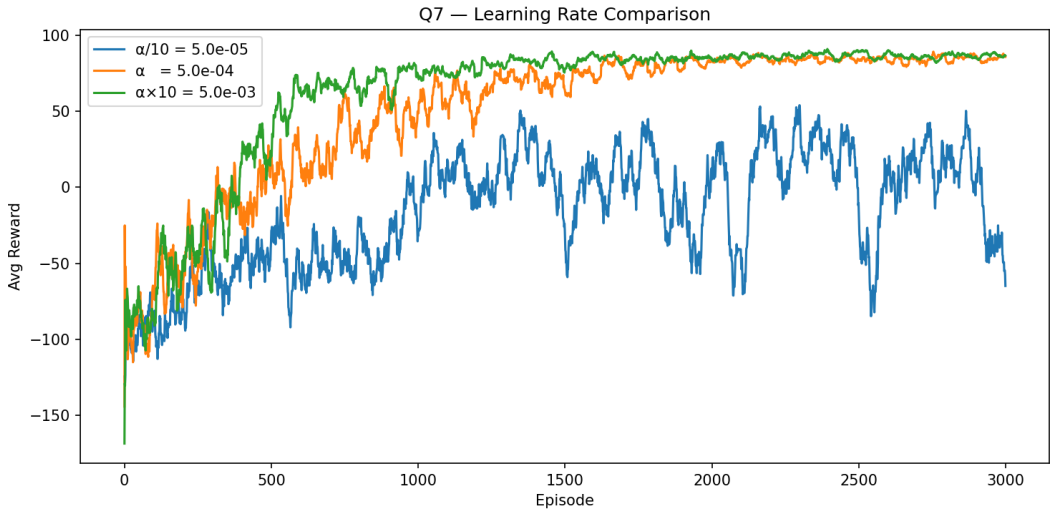


Figure 6: Standard DQN: effect of learning rate α on average reward.

Discussion: The slow rate $\alpha/10 = 5 \times 10^{-5}$ (blue) learns too slowly — after 3000 episodes it is still rising with high variance and has not converged. The baseline $\alpha = 5 \times 10^{-4}$ (orange) converges smoothly by episode ≈1000 and stabilizes at 85–90. The fast rate $\alpha \times 10 = 5 \times 10^{-3}$ (green) converges earliest (reward ≈70 by episode 300) but becomes highly unstable after episode 2000, with large oscillations and a sharp drop near episode 2500 caused by gradient steps overshooting the loss minimum. The baseline learning rate provides the best balance of speed and stability.

8 Q8: Double DQN

Standard DQN uses the same target network to both *select* and *evaluate* the best next action in the Bellman target ($\max_{a'} Q^{w^-}(s'_i, a')$). Because the max operator is biased upward in the presence of noisy Q-value estimates, this coupling leads to a systematic overestimation of Q-values. Double DQN decouples the two operations: the online Q-network Q^w is used to *select* the greedy action, and the target network Q^{w^-} is used to *evaluate* it. The Double DQN target is:

$$z_i = r_i + \gamma\, Q^{w^-}\left(s'_i,\ \arg\max_{a'} Q^w(s'_i, a')\right)(1 - \text{done}_i) \tag{1}$$

Code changes from Standard DQN. The only modification is inside the target computation in the agent's `update()` method. In the Double DQN branch, the next-state greedy action is first selected using the online network, $a^* = \arg\max_{a'} Q^w(s'_i, a')$, and this selected action is then evaluated by the target network to obtain $Q^{w^-}(s'_i, a^*)$. The network architecture, loss function, replay buffer, ϵ-greedy policy, and soft-update rule are all unchanged from Standard DQN.

Key code — Double DQN target computation: See Figure 7: `q_net` selects the greedy action $a^* = \arg\max_{a'} Q^w(s'_i, a')$, then `target_net` evaluates $Q^{w^-}(s'_i, a^*)$.

```python
# Target z_i
with torch.no_grad():
    if self.mode == 'double':
        # Double DQN: Q-net selects action, target-net evaluates
        best_a = self.q_net(s_next).argmax(dim=1, keepdim=True)
        q_next = self.target_net(s_next).gather(1, best_a).squeeze(1)
    else:
        q_next = self.target_net(s_next).max(dim=1)[0]

    targets = r + self.gamma * q_next * (1.0 - done)
```

Figure 7: Double DQN core update implementation.

Hyperparameters: No changes from Standard DQN.

8.1 Q8-2: Avg Reward, Avg Loss, and Avg Length

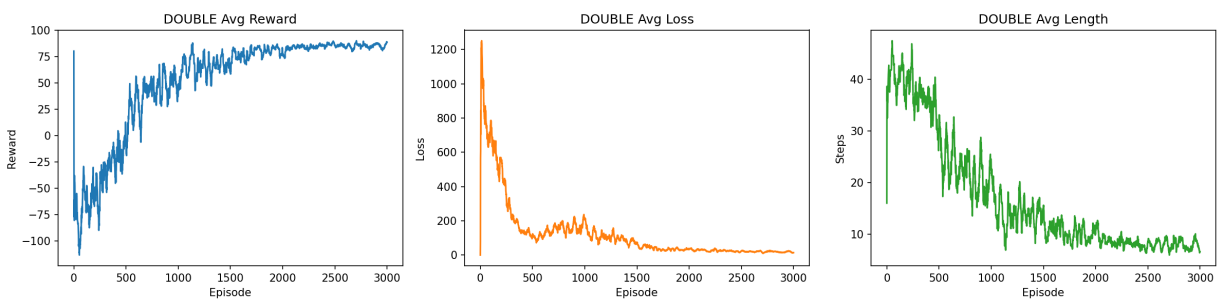


Figure 8: Double DQN: training curves.

Discussion: The Avg Reward rises from ≈ -100 and converges to $\approx$85–90, similar to Standard DQN but with slightly less variance in episodes 200–600. The Avg Loss peaks higher at $\approx$1200 versus $\approx$800 for Standard DQN. This larger initial loss is expected: This difference likely reflects run-to-run variability from random weight initialization and replay buffer seeding, since both methods share the same architecture, optimizer, and loss function. Both loss curves decay to comparable levels by episode 3000.

8.2 Q8-3: Obtained Policy

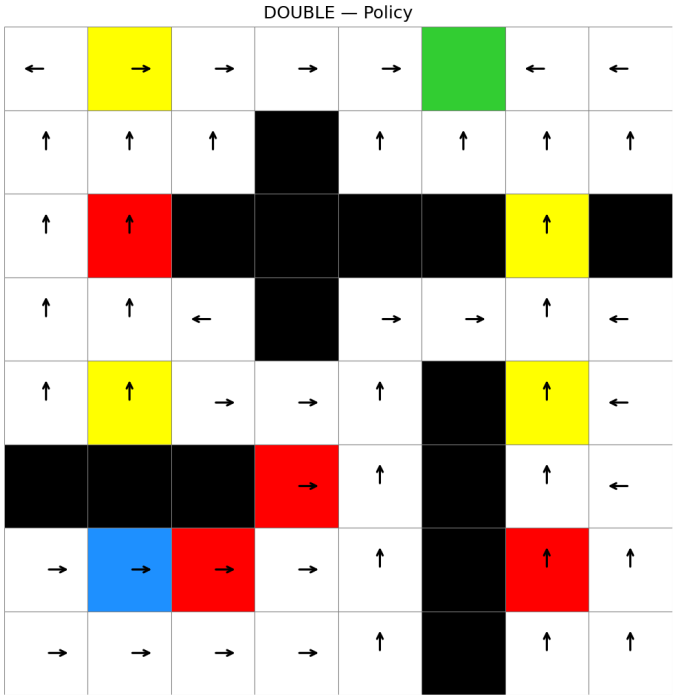


Figure 9: Double DQN: learned greedy policy.

Discussion: The learned policy is visually near-identical to Standard DQN — arrows route through the same corridor (column 4 upward, right to column 6, then left-up to the goal). Since both methods use ϵ-greedy exploration and the same network architecture, the converged policies differ only in Q-value accuracy, not in the discrete $\arg\max$ action per cell.

8.3 Q8-4: State Values

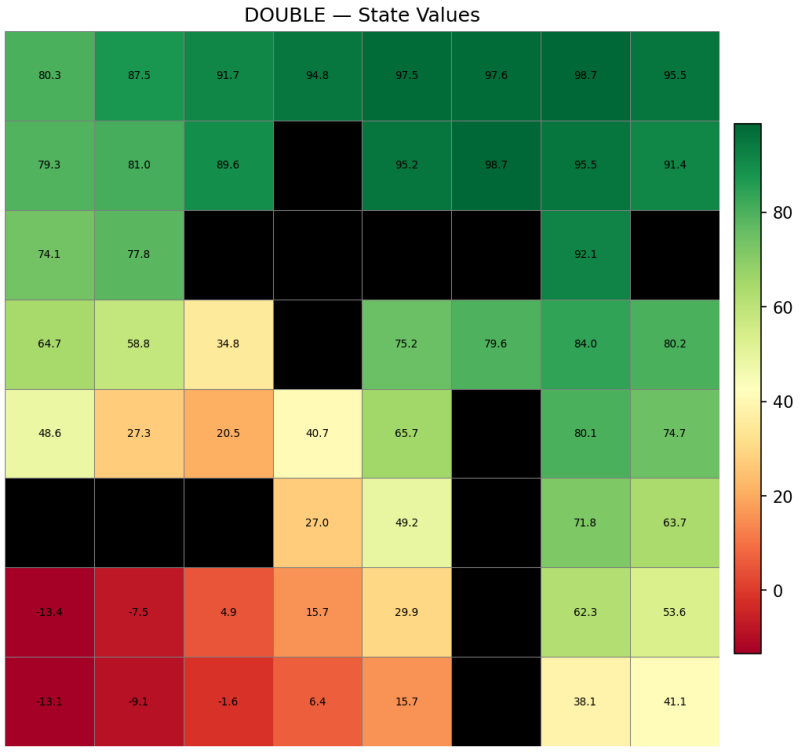


Figure 10: Double DQN: state values $V(s)$.

Discussion: Double DQN produces slightly lower state values in distant regions: the bottom-left corner reaches -13.4 at $(0, 6)$ versus -7.6 for Standard DQN. This is the expected effect of correcting overestimation bias — Standard DQN's max-operator inflates Q-values, while Double DQN's decoupled selection/evaluation yields more realistic estimates far from the goal. Values near the goal (row 0) remain similarly high (97–99) in both methods.

8.4 Q8-5: Path from Start

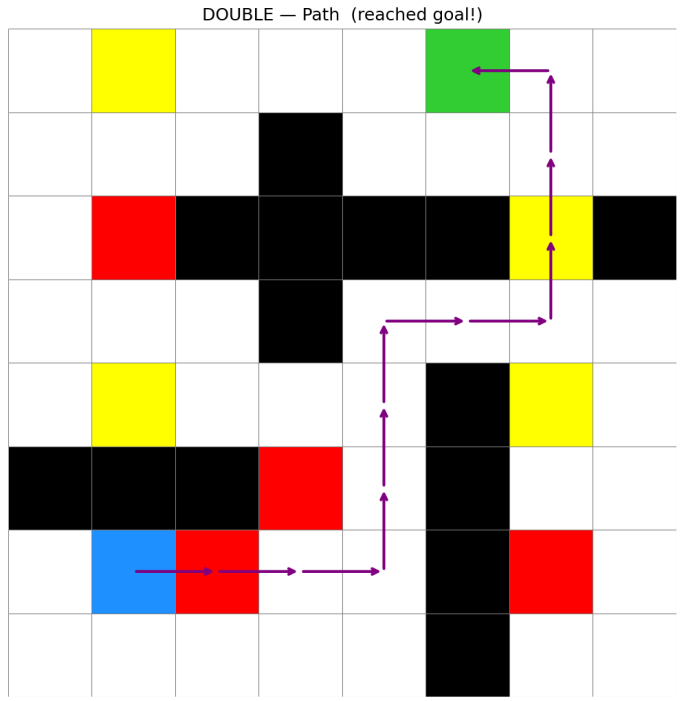


Figure 11: Double DQN: path from start to goal.

Discussion: The greedy path from (1, 6) follows the same main corridor as Standard DQN: right ×3 to (4, 6), up ×3 to (4, 3), right ×2 to (6, 3), up ×3 to (6, 0), left to (5, 0) — goal reached in 12 steps.

8.5 Q8-6: Avg Length

Discussion: The Avg Length for Double DQN decreases from ≈45 steps to 5–10 steps by episode 3000, matching Standard DQN's convergence speed. Both methods achieve near-identical final episode lengths, indicating that the overestimation correction of Double DQN improves value accuracy but does not alter path efficiency on this small maze.

Comparison with Standard DQN: The side-by-side training curves for Standard, Double, and Dueling DQN are shown in Figure 17 in Section 9; the Standard and Double DQN curves in that figure provide the direct comparison requested here.

9 Q9: Dueling DQN

The Q-function is decomposed as:

$$Q(s,a) = V(s) + A(s,a) - \frac{1}{|\mathcal{A}|}\sum_{a'} A(s,a') \tag{2}$$

where $V(s)$ is the state value stream and $A(s, a)$ is the advantage stream. The subtraction of the mean enforces the identifiability constraint: $V(s)$ captures the average value over actions, and $A(s, a)$ captures the deviation of each action from that average.

Key code — Dueling DQN architecture: See Figure 12: a shared feature trunk feeds two heads — `value_stream` outputting $V(s)$ and `advantage_stream` outputting $A(s, a)$ — combined as `v + a - a.mean(dim=-1, keepdim=True)`.

```python
def __init__(self, input_dim=2, output_dim=4, hidden_dim=128):
    super().__init__()
    self.feature = nn.Sequential(
        nn.Linear(input_dim, hidden_dim),   # 2 → 128
        nn.ReLU(),
        nn.Linear(hidden_dim, hidden_dim),  # 128 → 128
        nn.ReLU(),
    )
    self.value_stream = nn.Sequential(
        nn.Linear(hidden_dim, hidden_dim),
        nn.ReLU(),
        nn.Linear(hidden_dim, 1),
    )
    self.advantage_stream = nn.Sequential(
        nn.Linear(hidden_dim, hidden_dim),
        nn.ReLU(),
        nn.Linear(hidden_dim, output_dim),
    )

def forward(self, x):
    feat = self.feature(x)
    v = self.value_stream(feat)
    a = self.advantage_stream(feat)
    return v + a - a.mean(dim=-1, keepdim=True)
```

Figure 12: Dueling DQN network architecture implementation.

Code changes from Standard DQN. Only the network architecture is changed: the plain feed-forward `DQN` module is replaced by a `DuelingDQN` module that shares a feature-extraction trunk and then splits into two heads — a scalar value stream $V(s)$ and an advantage stream $A(s, a)$ — whose outputs are combined via the identifiability formula above to produce the final Q-values. The training loop, target computation, loss function, replay buffer, ϵ-greedy policy, and soft-update rule are all identical to Standard DQN; they simply operate on the Q-values produced by the new architecture.

Hyperparameters: No changes from Standard DQN.

9.1 Q9-2: Avg Reward, Avg Loss, and Avg Length

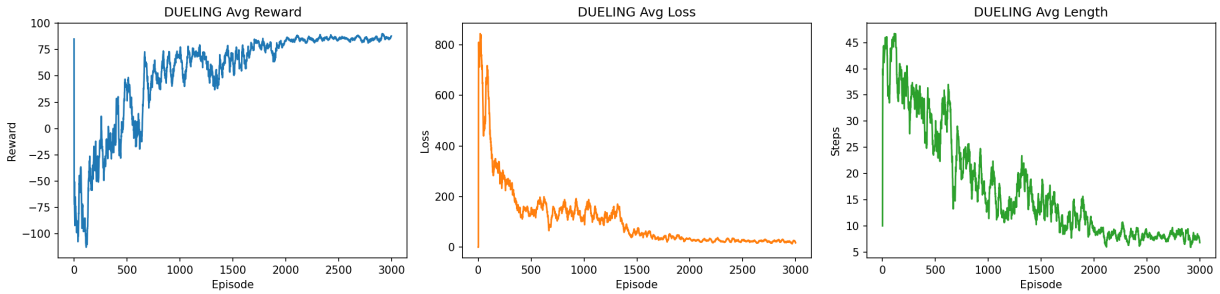


Figure 13: Dueling DQN: training curves.

Discussion: The Avg Reward converges faster than both previous methods: reward crosses 50 by episode ≈400 and reaches 85–90 by episode ≈1200. The Avg Loss peaks at ≈850 and decays similarly to Standard DQN. The Avg Length falls most rapidly, reaching 5–7 steps by episode 2000. The faster convergence is attributed to the value stream $V(s)$ learning a state-level baseline efficiently, allowing the advantage stream to refine action differences with less data.

9.2 Q9-3: Obtained Policy

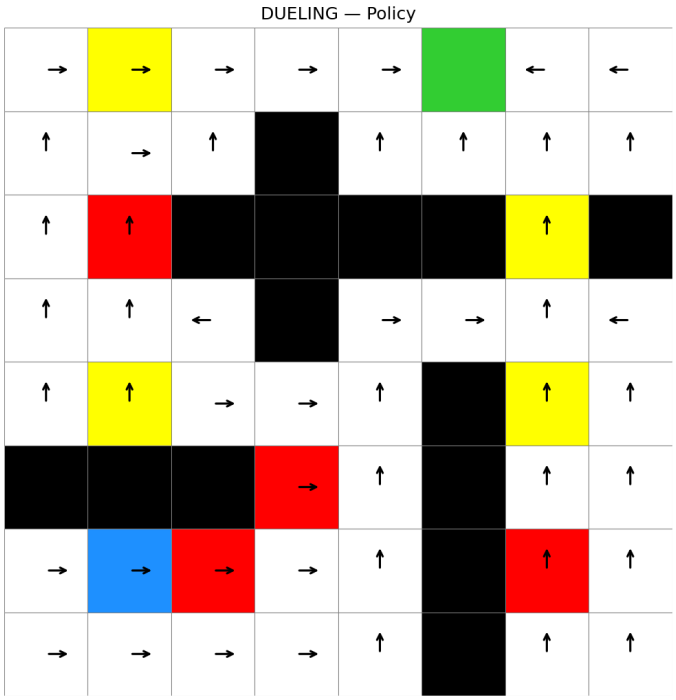


Figure 14: Dueling DQN: learned greedy policy.

Discussion: The policy is structurally the same as Standard and Double DQN — arrows correctly route toward the goal throughout the maze. The Dueling architecture does not change which

action is arg max optimal; it only changes how Q-values are computed internally via the $V + A - \bar{A}$ decomposition.

9.3 Q9-4: State Values

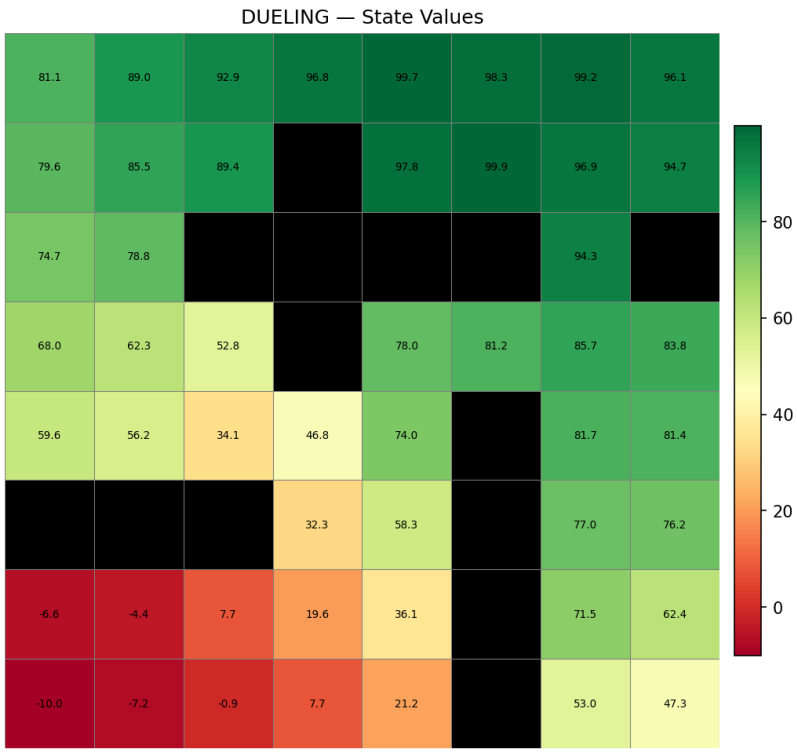


Figure 15: Dueling DQN: state values $V(s)$.

Discussion: Dueling DQN produces the highest values near the goal: $(4, 0)$ reaches 99.7 and $(5, 1)$ reaches 99.9. The bottom-left values are slightly more negative (-10.0 at $(0, 7)$) than Standard DQN (-6.4), consistent with the same conservative correction seen in Double DQN. The value gradient is sharper and more spatially coherent than the other two methods, as the dedicated $V(s)$ stream explicitly estimates state importance independent of action-level noise.

9.4 Q9-5: Path from Start

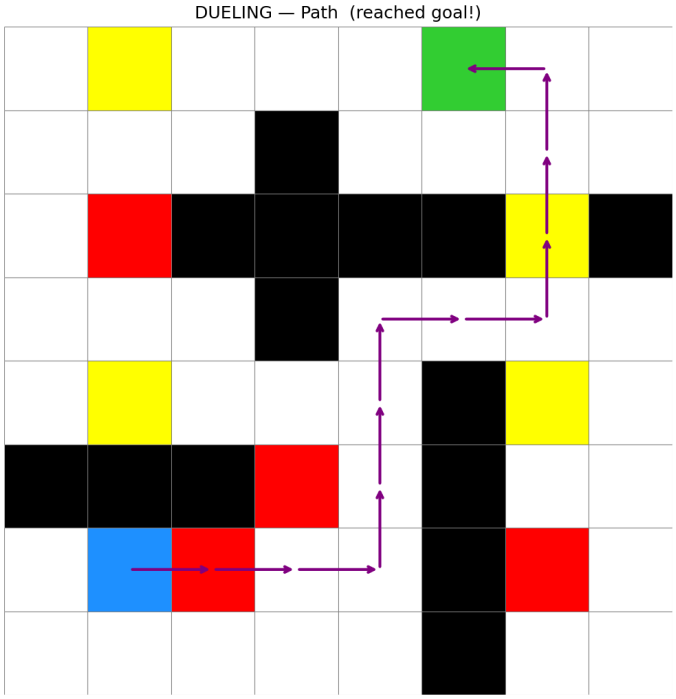
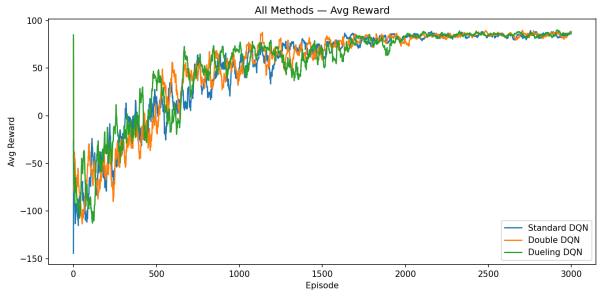


Figure 16: Dueling DQN: path from start to goal.

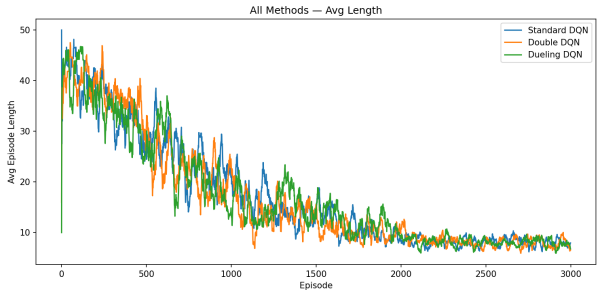
Discussion: The greedy path from $(1, 6)$ is identical to Double DQN: right $\times 3$, up $\times 3$, right $\times 2$, up $\times 3$, left — goal reached in $\approx$12 steps. All three methods converge to the same optimal corridor.

9.5 Q9-6: Avg Length

Discussion: The Avg Length for Dueling DQN decreases fastest among all three methods, stabilizing around 5–7 steps by episode 2000 versus 5–10 for Standard and Double DQN at episode 3000. This confirms that the value/advantage decomposition improves sample efficiency, enabling the agent to find short paths earlier in training.



(a) Avg Reward



(b) Avg Episode Length

Figure 17: Three-way comparison: Standard DQN vs Double DQN vs Dueling DQN.

Comparison with Standard and Double DQN:

Findings: All three DQN variants successfully solve the 8×8 stochastic maze, reaching the goal reliably and converging to similar final performance. The primary differences are in convergence speed and value accuracy: Dueling DQN is the most sample-efficient, Double DQN produces the most accurate (least overestimated) Q-values, and Standard DQN is the simplest baseline. On this small discrete task, the advantages of the advanced architectures are modest; their benefits would be more pronounced in larger state spaces where data efficiency and value accuracy are critical.

AI Assistance Disclosure: Generative AI tools (Claude) were used to assist with code development, including debugging, code structure suggestions, and understanding reinforcement learning concepts. All code was reviewed, understood, and tested by myself. The final implementation and analysis represent my own original work.